Problem Statement: QuakeWatch

Real-time earthquake intelligence with revision-aware records, alerts, and reproducible research exports

Draft for review (source-derived, 2026-10-10)

Status of this document: DRAFT. Every statement below is derived from the working copy (`D:\quakewatch`) and from `docs/PRD.md`. Items marked *(execution pending)* describe behaviour that is implemented in code but has not yet been demonstrated against a running development database.

1. Domain

Earthquake information is published continuously by public seismic agencies. The United States Geological Survey (USGS) publishes GeoJSON summary feeds, a Catalog API for historical searches, and per-event detail products. The data are authoritative, but they are fragmented across endpoints, and the same earthquake can change after its first publication: magnitude, depth, coordinates, review status, and attached products may be revised.

QuakeWatch is a web application that ingests these feeds, stores each earthquake as one evolving record in a document and graph database, and presents the data as maps, tables, charts, alerts, and downloadable datasets. It is not an earthquake prediction system, and it is not a replacement for official emergency-warning systems. Earthquake data are courtesy of the U.S. Geological Survey (USGS).

2. Problem

The problem has six parts, taken from the product requirements (`docs/PRD.md`, section 4):

Ref	Problem	Response in QuakeWatch
P1	Source data are fragmented across feeds, event details, and products	One event-centric record, normalised from the feeds
P2	Records change after first publication	Revision history is kept; revisions are not silently overwritten
P3	A map alone does not answer analytical questions	Map, explorer, charts, saved searches, and exports together
P4	Alerts are opaque	Deterministic rules; each match stores an explanation
P5	Research datasets are hard to reproduce	Exports carry a manifest with query, source, time, schema version, and checksum
P6	Users cannot see how trustworthy a value is	Source values and derived values are kept separate; timestamps are shown

A document and graph database suits this problem because the data have three shapes that a single relational table design handles poorly: one document per event with nested provenance, a sequence of revisions per event, and relationships between events, regions, networks, users, and alert rules. ArangoDB stores all three in one system: documents, edge collections for graph traversal, and an ArangoSearch view for text search.

3. Objectives

1. Ingest the USGS summary feed on a schedule, and backfill historical events from the Catalog API within bounded limits (FR-01, FR-02).
2. Store one canonical record per source event, with idempotent upserts keyed on source and source event ID (FR-03).

3. Record meaningful revisions and provenance for every sourced event (FR-04, FR-05).
4. Provide filters, a viewport-aware map, event detail pages, and analytics (FR-06 to FR-09).
5. Let registered users create alert rules, see explainable matches, and save searches (FR-10 to FR-12).
6. Export research datasets with a manifest and a checksum (FR-13).
7. Give administrators an ingestion-health dashboard and an audit trail for privileged actions (FR-14, FR-15).

4. Scope

In scope (version 1 MVP, per `docs/PRD.md` section 6): USGS summary-feed ingestion; historical backfill through the Catalog API; ArangoDB event storage; revision history; interactive map; event explorer; event detail; analytics dashboard; user accounts; saved searches; user-defined alert rules; administrator ingestion dashboard; CSV and JSON research export.

Out of scope: earthquake prediction; official emergency warnings; alerts delivered outside the application (for example, SMS or e-mail); non-USGS sources beyond the configured adapters.

5. Users

User	Needs	Pages in the working copy
Students and researchers	Explore events; build reproducible datasets; export CSV and JSON	Explorer, Analytics, Research
Disaster-management learners and analysts	Monitor recent significant events; save watch areas and threshold rules	Dashboard, Map, Alerts
General users	Understand what happened, where, and how strong, without an API	Dashboard, Event detail, About

User	Needs	Pages in the working copy
Administrators	Monitor ingestion health, failures, stale data, and audit logs	Status, Operations (staff only)

6. Expected value

- A single authoritative record per earthquake, with its history, rather than a snapshot that is overwritten.
- Alerts whose matching logic can be read and checked: each match states the rule conditions that held.
- Research exports that can be re-created: the manifest records the query, retrieval time, source, schema version, and a SHA-256 checksum of the rows.
- A working example of a document and graph database applied to a real, changing public data set.

7. Limits stated plainly

- QuakeWatch depends on the availability and content of the USGS feeds. Feed freshness is shown in the application.
- The application does not predict earthquakes and does not replace official alerting.
- The academic demonstration requires a running ArangoDB instance and a populated development database. Record counts, query outputs, and performance measurements for the current code are *pending* until that database is available.